

Definition

Seien $u, v \in (V \cup \Sigma)^*$. Wir schreiben $u \Rightarrow v$ (u **leitet** v **ab**), falls $u = xAy$ und $v = xwy$ für eine Regel $A \rightarrow w \in R$.

$u \Rightarrow^* v$: v ist aus u in null oder mehr Schritten ableitbar.

Definition: Erzeugte Sprache

Die von G **erzeugte Sprache** ist:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

Definition: Kontextfreie Sprache

Eine Sprache heißt **kontextfrei** (context-free), wenn es eine CFG gibt, die sie erzeugt.

Grammatik G_3

$$S \rightarrow aSb \mid SS \mid \varepsilon$$

$L(G_3)$ = alle Zeichenketten korrekt verschachtelter Klammern (interpretiere a als "(" und b als ")").

Grammatik G_3

$$S \rightarrow aSb \mid SS \mid \varepsilon$$

$L(G_3)$ = alle Zeichenketten korrekt verschachtelter Klammern (interpretiere a als "(" und b als ")").

Ableitung von $aabb$:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$$

Ableitung von $abab$:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$$

Grammatik G_3

$$S \rightarrow aSb \mid SS \mid \varepsilon$$

$L(G_3)$ = alle Zeichenketten korrekt verschachtelter Klammern (interpretiere a als "(" und b als ")").

Ableitung von $aabb$:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$$

Ableitung von $abab$:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$$

Beachte: Das ist die **Dyck-Sprache** – eine der wichtigsten kontextfreien Sprachen.

Grammatik G_4 (Sipser)

$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle &\rightarrow \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\ \langle \text{FACTOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$

Ableitung von $a + a \times a$:

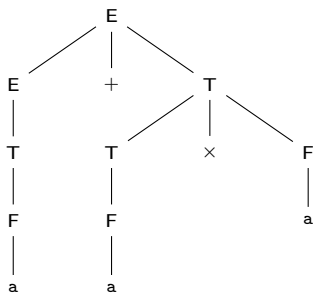
$$\begin{aligned}\langle \text{EXPR} \rangle &\Rightarrow \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \Rightarrow \langle \text{TERM} \rangle + \langle \text{TERM} \rangle \Rightarrow \langle \text{FACTOR} \rangle + \langle \text{TERM} \rangle \\ &\Rightarrow a + \langle \text{TERM} \rangle \Rightarrow a + \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \\ &\Rightarrow a + \langle \text{FACTOR} \rangle \times \langle \text{FACTOR} \rangle \Rightarrow a + a \times a\end{aligned}$$

Diese Grammatik codiert **Operatorprioritäten**: $\times$ bindet stärker als $+$.

Ableitungsbäume (Parse Trees)

Ein **Ableitungsbaum** stellt eine Ableitung graphisch dar:

Grammatik G_4 , Ableitung von $a + a \times a$:



- Wurzel = Startvariable
- Innere Knoten = Variablen
- Blätter = Terminale oder ε
- Blätter von links nach rechts gelesen = abgeleitete Zeichenkette

Linksableitung und Rechtsableitung

Definition

- **Linksableitung**: In jedem Schritt wird die **linkeste** Variable ersetzt.
- **Rechtsableitung**: In jedem Schritt wird die **rechteste** Variable ersetzt.

Beispiel mit $G_3: S \rightarrow aSb \mid SS \mid \varepsilon$. Ableitung von $abab$:

Linksableitung:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$$

Rechtsableitung:

$$S \Rightarrow SS \Rightarrow SaSb \Rightarrow Sab \Rightarrow aSbab \Rightarrow abab$$

Linksableitung und Rechtsableitung

Definition

- **Linksableitung**: In jedem Schritt wird die **linkeste** Variable ersetzt.
- **Rechtsableitung**: In jedem Schritt wird die **rechteste** Variable ersetzt.

Beispiel mit $G_3: S \rightarrow aSb \mid SS \mid \varepsilon$. Ableitung von $abab$:

Linksableitung:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$$

Rechtsableitung:

$$S \Rightarrow SS \Rightarrow SaSb \Rightarrow Sab \Rightarrow aSbab \Rightarrow abab$$

Beide Ableitungen ergeben den *gleichen* Ableitungsbaum. Jeder Ableitungsbaum hat genau eine Linksableitung und eine Rechtsableitung.

Definition 2.7 (Sipser)

Eine Zeichenkette w wird von einer CFG G **mehrdeutig abgeleitet**, wenn es **zwei oder mehr verschiedene Ableitungsbäume** gibt, die w erzeugen.

Eine CFG ist **mehrdeutig**, wenn sie mindestens eine Zeichenkette mehrdeutig ableitet.

Definition 2.7 (Sipser)

Eine Zeichenkette w wird von einer CFG G **mehrdeutig abgeleitet**, wenn es **zwei oder mehr verschiedene Ableitungsbäume** gibt, die w erzeugen.

Eine CFG ist **mehrdeutig**, wenn sie mindestens eine Zeichenkette mehrdeutig ableitet.

Beispiel: Mehrdeutige Grammatik G_5

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid a$$

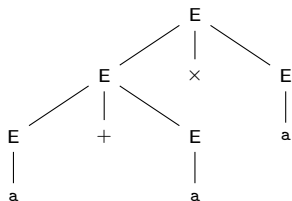
Die Zeichenkette $a + a \times a$ hat **zwei** Ableitungsbäume – einen, der $+$ zuerst anwendet und einen, der $\times$ zuerst anwendet.

Dagegen: Die Grammatik G_4 (mit EXPR, TERM, FACTOR) ist *nicht* mehrdeutig – sie codiert die Prioritäten in der Struktur.

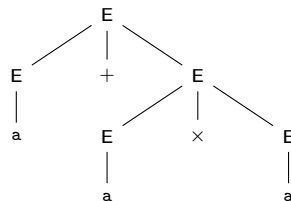
Mehrdeutigkeit – die zwei Parse Trees für G_5

$G_5: E \rightarrow E + E \mid E \times E \mid a.$ Zeichenkette: $a + a \times a$.

Baum 1: $(a + a) \times a$



Baum 2: $a + (a \times a)$



Zwei verschiedene Ableitungsbäume $\Rightarrow G_5$ ist **mehrdeutig**.

Für einen Compiler wäre das problematisch, da die Semantik unklar ist!

Inhärente Mehrdeutigkeit

Manche Sprachen besitzen die Eigenschaft, dass *jede* Grammatik, die sie erzeugt, mehrdeutig ist.

Definition

Eine kontextfreie Sprache heißt **inhärent mehrdeutig**, wenn jede CFG, die sie erzeugt, mehrdeutig ist.

Inhärente Mehrdeutigkeit

Manche Sprachen besitzen die Eigenschaft, dass *jede* Grammatik, die sie erzeugt, mehrdeutig ist.

Definition

Eine kontextfreie Sprache heißt **inhärent mehrdeutig**, wenn jede CFG, die sie erzeugt, mehrdeutig ist.

Beispiel

Die Sprache

$$L = \{a^i b^j c^k \mid i = j \text{ oder } j = k\}$$

ist inhärent mehrdeutig. Für Wörter der Form $a^n b^n c^n$ muss jede Grammatik “entscheiden”, ob sie $i = j$ oder $j = k$ matcht, und erzeugt so zwei Ableitungsbäume.

Bemerkung: Nicht jede mehrdeutige Grammatik erzeugt eine inhärent mehrdeutige Sprache – oft gibt es eine äquivalente, eindeutige Grammatik (wie G_4 statt G_5).

Chomsky-Normalform (CNF)

Definition 2.8 (Sipser)

Eine CFG ist in **Chomsky-Normalform**, wenn jede Regel eine der folgenden Formen hat:

- $A \rightarrow BC$ (zwei Variablen)
- $A \rightarrow a$ (ein Terminal)
- $S \rightarrow \varepsilon$ (nur für die Startvariable, falls $\varepsilon \in L$)

wobei $A, B, C \in V$, $B \neq S$, $C \neq S$, und $a \in \Sigma$.

Chomsky-Normalform (CNF)

Definition 2.8 (Sipser)

Eine CFG ist in **Chomsky-Normalform**, wenn jede Regel eine der folgenden Formen hat:

- $A \rightarrow BC$ (zwei Variablen)
- $A \rightarrow a$ (ein Terminal)
- $S \rightarrow \varepsilon$ (nur für die Startvariable, falls $\varepsilon \in L$)

wobei $A, B, C \in V$, $B \neq S$, $C \neq S$, und $a \in \Sigma$.

Warum CNF?

- Jeder Ableitungsbaum ist ein **Binärbaum** (Schlüssel für das Pumping-Lemma!)
- Ermöglicht den **CYK-Algorithmus** für Parsing in $O(n^3 \cdot |G|)$
- Vereinfacht viele Beweise über kontextfreie Sprachen

Satz 2.9 (Sipser)

Jede kontextfreie Sprache wird von einer Grammatik in Chomsky-Normalform erzeugt.

Satz 2.9 (Sipser)

Jede kontextfreie Sprache wird von einer Grammatik in Chomsky-Normalform erzeugt.

Beweis (Konstruktion):

Die Umwandlung erfolgt in vier Schritten:

- ➊ Neue Startvariable $S_0 \rightarrow S$ einführen (damit S nicht rechts vorkommt).
- ➋ **ε -Regeln eliminieren**: Für jede Regel $A \rightarrow \varepsilon$ (mit $A \neq S_0$) alle Vorkommen von A in rechten Seiten optional machen.
- ➌ **Kettenregeln eliminieren**: Regeln $A \rightarrow B$ ersetzen (transitive Auflösung).
- ➍ **Regeln der Form $A \rightarrow u_1 u_2 \cdots u_k$ anpassen**:
 - Terminale a durch neue Variablen $U_a \rightarrow a$ ersetzen.
 - Lange rechte Seiten ($k \geq 3$) kaskadenartig aufspalten.

Grammatik: $S \rightarrow ASA \mid aB$, $A \rightarrow B \mid S$, $B \rightarrow b \mid \varepsilon$.

CNF-Umwandlung – Beispiel

Grammatik: $S \rightarrow ASA \mid aB$, $A \rightarrow B \mid S$, $B \rightarrow b \mid \varepsilon$.

Schritt 1 (neue Startvariable): $S_0 \rightarrow S$.

Schritt 2 (ε -Regeln, hier $B \rightarrow \varepsilon$): Überall, wo B rechts vorkommt, Varianten mit und ohne B hinzufügen.

- $S \rightarrow ASA \mid aB \mid a$ (aB ohne B : a)
- $A \rightarrow B \mid S \mid \varepsilon$ (A wird nun auch nullable)

Dann $A \rightarrow \varepsilon$ eliminieren: $S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$.

CNF-Umwandlung – Beispiel

Grammatik: $S \rightarrow ASA \mid aB$, $A \rightarrow B \mid S$, $B \rightarrow b \mid \varepsilon$.

Schritt 1 (neue Startvariable): $S_0 \rightarrow S$.

Schritt 2 (ε -Regeln, hier $B \rightarrow \varepsilon$): Überall, wo B rechts vorkommt, Varianten mit und ohne B hinzufügen.

- $S \rightarrow ASA \mid aB \mid a$ (aB ohne B : a)
- $A \rightarrow B \mid S \mid \varepsilon$ (A wird nun auch nullable)

Dann $A \rightarrow \varepsilon$ eliminieren: $S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$.

Schritt 3 (Kettenregeln $S_0 \rightarrow S$, $A \rightarrow B$, $A \rightarrow S$, $S \rightarrow S$ eliminieren):

Transitive Auflösung.

Schritt 4 (Binär machen): $S \rightarrow ASA$ wird $S \rightarrow AA_1$, $A_1 \rightarrow SA$.

Terminale: $S \rightarrow U_a B$ mit $U_a \rightarrow a$.

$\Rightarrow$ Alle Regeln in CNF.



Beobachtung

In einer CNF-Grammatik hat jeder Ableitungsbaum für ein Wort w mit $|w| = n \geq 1$ genau $2n - 1$ innere Knoten (Variablen-Anwendungen).

Grund:

- Jede Regel $A \rightarrow BC$ erzeugt genau **zwei** Kinder (binärer Baum).
- Jede Regel $A \rightarrow a$ erzeugt genau **ein** Blatt.
- Ein Binärbaum mit n Blättern hat $n - 1$ innere Verzweigungsknoten.

Beobachtung

In einer CNF-Grammatik hat jeder Ableitungsbaum für ein Wort w mit $|w| = n \geq 1$ genau $2n - 1$ innere Knoten (Variablen-Anwendungen).

Grund:

- Jede Regel $A \rightarrow BC$ erzeugt genau **zwei** Kinder (binärer Baum).
- Jede Regel $A \rightarrow a$ erzeugt genau **ein** Blatt.
- Ein Binärbaum mit n Blättern hat $n - 1$ innere Verzweigungsknoten.

Folgerung für das Pumping-Lemma: Wenn der Ableitungsbaum *hoch genug* ist, muss sich auf einem Pfad von der Wurzel zu einem Blatt eine Variable wiederholen.

1 2.1 Kontextfreie Grammatiken

- Motivation und Definition
- Ableitungsbäume und Mehrdeutigkeit
- Chomsky-Normalform

2 2.2 Kellerautomaten

- Definition und Beispiele
- Äquivalenz von CFG und PDA
- Deterministische Kellerautomaten

3 2.3 Nichtkontextfreie Sprachen

- Das Pumping-Lemma für kontextfreie Sprachen
- Anwendungen des Pumping-Lemmas
- Abschlusseigenschaften kontextfreier Sprachen
- Zusammenfassung

Kellerautomaten – Motivation

Erinnerung: Endliche Automaten haben nur *endlich viel* Speicher.

Ein **Kellerautomat** (Pushdown Automaton, PDA) erweitert den NFA um einen **Stack** (Keller):

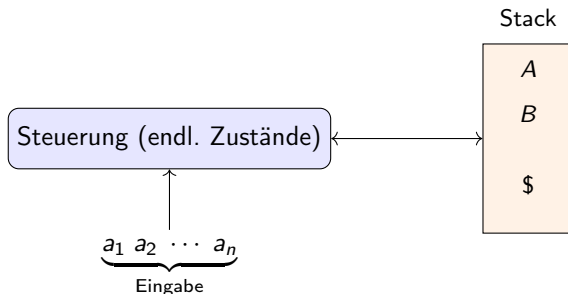
- Unbeschränkter Speicher nach dem **LIFO**-Prinzip (Last In, First Out)
- Der Automat kann Symbole auf den Stack **pushen** und vom Stack **poppen**
- Nur das **oberste** Stacksymbol ist sichtbar

Kellerautomaten – Motivation

Erinnerung: Endliche Automaten haben nur *endlich viel* Speicher.

Ein **Kellerautomat** (Pushdown Automaton, PDA) erweitert den NFA um einen **Stack** (Keller):

- Unbeschränkter Speicher nach dem **LIFO**-Prinzip (Last In, First Out)
- Der Automat kann Symbole auf den Stack **pushen** und vom Stack **poppen**
- Nur das **oberste** Stacksymbol ist sichtbar



Definition 2.13 (Sipser)

Ein **Kellerautomat** (PDA) ist ein 6-Tupel $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- 1 Q – endliche Zustandsmenge
- 2 Σ – Eingabealphabet
- 3 Γ – **Stackalphabet**
- 4 $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ – Übergangsfunktion
- 5 $q_0 \in Q$ – Startzustand
- 6 $F \subseteq Q$ – akzeptierende Zustände

Dabei: $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ und $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$.

Definition 2.13 (Sipser)

Ein **Kellerautomat** (PDA) ist ein 6-Tupel $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- 1 Q – endliche Zustandsmenge
- 2 Σ – Eingabealphabet
- 3 Γ – **Stackalphabet**
- 4 $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ – Übergangsfunktion
- 5 $q_0 \in Q$ – Startzustand
- 6 $F \subseteq Q$ – akzeptierende Zustände

Dabei: $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ und $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$.

$\delta(q, a, s) \ni (r, t)$ bedeutet: Im Zustand q , bei Eingabe a , mit s oben auf dem Stack $\Rightarrow$ gehe nach r und ersetze s durch t auf dem Stack.

($a = \varepsilon$: nichts lesen; $s = \varepsilon$: nichts poppen; $t = \varepsilon$: nichts pushen.)

Schreibweise auf Kanten:

$$a, b \rightarrow c$$

bedeutet: Lies a von der Eingabe, poppe b vom Stack, pushe c auf den Stack.

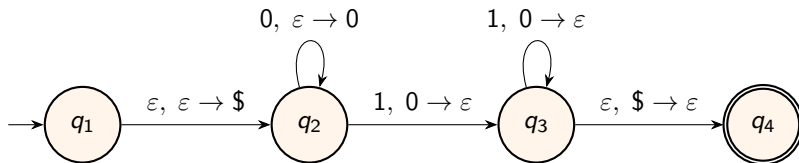
Spezialfälle:

- $a, \varepsilon \rightarrow c$: Lies a , poppe nichts, pushe c
- $\varepsilon, b \rightarrow \varepsilon$: Lies nichts, poppe b , pushe nichts
- $\varepsilon, \varepsilon \rightarrow c$: Lies nichts, poppe nichts, pushe c

Wichtig: PDAs sind **nichtdeterministisch** – es kann mehrere Übergänge geben. Akzeptanz: mindestens ein Berechnungspfad endet in F .

PDA – Beispiel 1

PDA für $L = \{0^n 1^n \mid n \geq 0\}$:

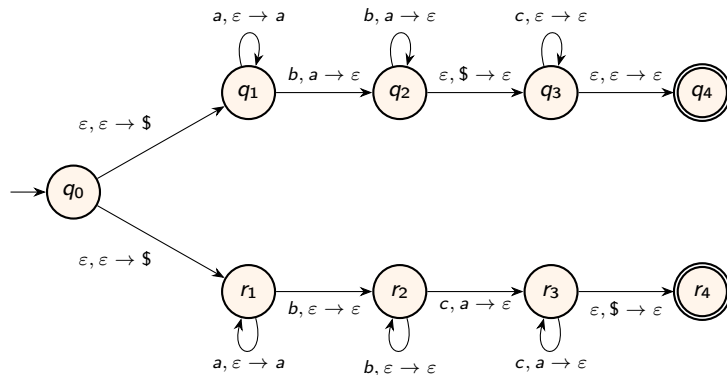


Arbeitsweise:

- 1 Pushe Bodenmarker $\$$.
- 2 Phase 1 (q_2): Für jede 0: pushe 0 auf den Stack.
- 3 Phase 2 (q_3): Für jede 1: poppe eine 0 vom Stack.
- 4 Akzeptiere (q_4), wenn $\$$ oben liegt und Eingabe leer.

PDA – Beispiel 2

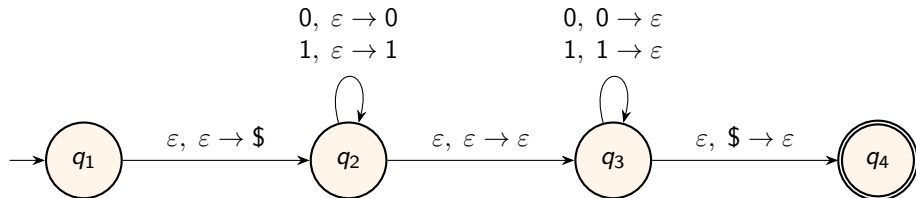
PDA für $L = \{a^i b^j c^k \mid i = j \text{ oder } i = k\}$:



Nichtdeterminismus ist hier entscheidend: Der PDA “rät”, ob $i = j$ oder $i = k$ gelten wird, und verfolgt den entsprechenden Zweig.

PDA – Beispiel 3

PDA für $L = \{ww^R \mid w \in \{0,1\}^*\}$ (gerade Palindrome):



Idee:

- 1 Pushe die erste Hälfte auf den Stack.
- 2 Rate nichtdeterministisch die Mitte.
- 3 Vergleiche die zweite Hälfte mit dem Stack (pop und vergleiche).